

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf\_5de56b64-647\agent-ab8032e15b0bfaa8e.jsonl`
- SHA-256: `d17812c43931533bee3b849436c449b56034f575e2e659071478ed73d1af3b5a`
- Source modified: `2026-06-21T14:40:05+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf\_5de56b64-647`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`

Transcript

1. user (2026-06-21T14:36:44.490Z)

Repo: C:\Users\avidu\Projects\badminton-highlight-indexer (engine) + C:\Users\avidu\Projects\kshelsutra (SPA). For an ALPHA-SHAREABLE deployment the SPA (hosted at some URL) calls the engine (hosted at another URL) cross-origin, gated by Cloudflare Access edge-auth (M9, merged #290, default-OFF). Investigate and cite file:line:

- Does the FastAPI app (backend/main.py) set up CORS middleware? Is the allowed-origins list configurable/env-driven, or absent? A cross-origin hosted SPA needs CORS. What's the gap?
- Read the M9 edge-auth code (grep for edge_auth / Cf-Access / owner_id / the auth middleware in backend/). What does flipping security.edge_auth_enabled require operationally (CF team domain, AUD, ingress lock) vs in code? Cite the config knobs.
- Does the SPA send any auth header / credentials, or rely on the CF Access cookie/redirect being transparent? Cite the SPA api client.
- Is there any CSRF / allowed_video_root / hosted-mode interaction that bites when the engine is public? Cite.

Classify: which pieces are claude-doable-no-spend code (CORS config, env-driven origins, SPA API-base) vs owner-gated config (CF team domain/AUD, the actual flip + deploy).

2. assistant (2026-06-21T14:36:46.661Z)

I'll systematically investigate the CORS setup, edge-auth implementation, and SPA-to-engine communication for this alpha-shareable deployment scenario.

3. user (2026-06-21T14:36:47.403Z)

Exit code 1
/usr/bin/bash: line 1: cd: C:\Users\avidu\Projects\badminton-highlight-indexer: No such file or directory

4. assistant (2026-06-21T14:36:49.077Z)

Let me correct the path syntax for Windows:

5. user (2026-06-21T14:36:50.707Z)

```
./claude/worktrees/agent-a771ff1d85e18f2ed/backend/main.py
./claude/worktrees/bhi-execpolicy/backend/main.py
./claude/worktrees/boxhill17/backend/main.py
./claude/worktrees/gap-doc/backend/main.py
./claude/worktrees/hardchop/backend/main.py
./claude/worktrees/pr-stitch/backend/main.py
./claude/worktrees/recdoc/backend/main.py
./claude/worktrees/resweep/backend/main.py
./claude/worktrees/tier1-windowing/backend/main.py
```

```
../.claude/worktrees/w2-safeguard/backend/main.py
./backend/main.py
```

6. assistant (2026-06-21T14:36:52.693Z)

Good. Now let me read the main FastAPI app and search for CORS and edge-auth:

7. user (2026-06-21T14:36:53.685Z)

```
1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException, Request
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )
39    from backend.results import Failure # standardized error/result contract
40    from backend.reporting import emit_indexer_report
41    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42    from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43    from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44
45    app = FastAPI(title="Sports Highlight Indexer API")
46
47
48    def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
49        """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
50        default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
```

```

51         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
52         src = os.environ if env is None else env
53         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
54         origins = [o.strip() for o in raw.split(",") if o.strip()]
55         return origins or ["*"]
56
57
58     # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
59     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
60     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
61     # lock it to its origin; the middleware is fixed at startup.
62     app.add_middleware(
63         CORSMiddleware,
64         allow_origins=_cors_origins_from_env(),
65         allow_credentials=False,
66         allow_methods=["*"],
67         allow_headers=["*"],
68     )
69
70     # Serves static frontend files directly (now under api/ per approved structure)
71     os.makedirs("backend/api/static", exist_ok=True)
72     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
73
74     @app.get("/", response_class=HTMLResponse)
75     def serve_index():
76         index_path = "backend/api/static/index.html"
77         if os.path.exists(index_path):
78             with open(index_path, "r", encoding="utf-8") as f:
79                 return f.read()
80         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
81
82     # Global variables initialized at startup
83     db_instance: Optional[Database] = None
84     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
85
86     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
87     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
88     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
89     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
90     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
91     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
92     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
93     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
94         JobWaiting,
95         _arm_wake_timer,
96         _drain_due_jobs,
97         _get_executor,
98         _job_to_request,
99         _MAX_DRAIN_WAKE_SEC,
100         _maybe_record_job_cost,
101         _run_claimed_job,
102         _schedule_next_drain_if_waiting,
103     )
104
105     # Legacy thin wrapper for any remaining direct calls during transition.
106     # New code should import load_app_config / AppConfig from backend.config directly.
107     def load_config(config_path: str = "config.json") -> Dict[str, Any]:

```

```

108         cfg = load_app_config(config_path)
109         return cfg.model_dump(mode="json")
110
111     # --- API Models ---
112     class ProcessRequest(BaseModel):
113         video_path: str
114         player_pool: Optional[List[str]] = None
115         max_frames: Optional[int] = None
116         segmenter: Optional[str] = None
117
118     class QueryRequest(BaseModel):
119         video_id: str
120         server: Optional[str] = None
121         receiver: Optional[str] = None
122         duration_min: Optional[float] = None
123         event_type: Optional[str] = None
124
125     class CompileRequest(BaseModel):
126         video_id: str
127         segment_ids: List[int]
128         output_filename: str
129
130     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
131         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
132
133         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
134         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
135         and
136         keep the prior good results intact ? a failed/empty re-run never destroys prior
137         data.
138         """
139         if segments:
140             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
141         else:
142             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
143
144     # --- API Endpoints ---
145     @app.get("/api/config")
146     def get_config():
147         return global_config
148
149     @app.get("/api/health")
150     def health():
151         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
152         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
153         confirm the engine is reachable and which detector is wired. Never raises."""
154         sport = getattr(global_config, "sport", None)
155         indexing = getattr(global_config, "indexing", None)
156         return {
157             "status": "ok",
158             "sport": sport,
159             "default_segmenter": getattr(indexing, "default_segmenter", None),
160         }
161
162     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
163         """The full hash ? validate ? segment ? report pipeline for one video.
164
165         This is the former synchronous /api/process body, unchanged in behavior, now run on
166         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
167         the sync endpoint used to return (UI compatibility); the per-video observability
168         report is still emitted in `finally:` and grafted as response["reports"].
169
170         `progress` is an optional callable(stage: str) used to surface coarse job progress.
171         Raises on unexpected internal errors ? the caller records those as a job failure

```

```

171         (after this function has already restored the pre-run segments, see below).
172         """
173         notify = progress or (lambda stage: None)
174
175         notify("hashing")
176         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
177         (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
178         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
179         # consumers (hash / validators / segmenter) use the resolved local path.
180         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
181         video_id = compute_video_id(local_video)
182         was_reingest = db_instance.get_video(video_id) is not None
183
184         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
185         notify("validating")
186         logger.info(f"Running validations for {req.video_path}...")
187         val_results, val_context = registry.run_all_with_context(local_video, global_config)
188         failures = [r for r in val_results if not r.passed]
189
190         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
191         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
192         seg_name = req.segmenter or default_seg
193         segmenter_actual = None
194         segmentation_ran = False
195         response: Optional[Dict[str, Any]] = None
196         try:
197             if failures:
198                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
199                 # status; it no longer destroys the video's prior good segments.
200                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
201                 response = {
202                     "video_id": video_id,
203                     "status": "failed",
204                     "message": "Video failed validation checks.",
205                     "results": [r.model_dump() for r in val_results],
206                 }
207                 return response
208
209             # 2. Run segmenter & indexer
210             logger.info("[API] Video passed checks. Segmenting and indexing...")
211             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
212
213             segmenter_cls = segmenter_registry.get(seg_name)
214             if not segmenter_cls:
215                 # Submit-time validation already 400s unknown names; this is the defensive
216                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
217                 available = list(segmenter_registry.list_available().keys())
218                 response = {
219                     "video_id": video_id,
220                     "status": "failed",
221                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
222                     "results": [r.model_dump() for r in val_results],
223                     "play_segments": [],
224                 }
225                 return response

```

```

226
227     logger.info(f"[API] Using segmenter: {seg_name}")
228     notify(f"segmenting ({seg_name})")
229     segmenter_actual = seg_name
230     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
231     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
232     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
233     segmenter = segmenter_cls(db_instance, global_config)
234     try:
235         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
236     except Exception:
237         # A raising segmenter must not leave half-written rows: restore the pre-run
238         # state (same destroy-after-confirm contract as the Failure branch), then
let
239         # the job worker record the failure.
240         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
241         raise
242     segmentation_ran = True
243
244     if isinstance(result, Failure):
245         logger.error(f"[API] Segmenter failed: {result.message}")
246         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
247         response = {
248             "video_id": video_id,
249             "status": "failed",
250             "message": f"Segmenter '{seg_name}' failed: {result.message}",
251             "results": [r.model_dump() for r in val_results],
252             "play_segments": [],
253         }
254         return response
255
256     segments = result.value if hasattr(result, "value") else result
257     notify("finalizing")
258     _finalize_reingest(video_id, segments, play_wm, cand_wm)
259
260     response = {
261         "video_id": video_id,
262         "status": "success",
263         "message": f"Successfully indexed {len(segments)} play segments using
'{{seg_name}}' segmenter.",
264         "results": [r.model_dump() for r in val_results],
265         "play_segments": segments,
266     }
267     return response
268     finally:
269         # Always emit the per-video observability report (next to the video, or to
270         # report.output_dir). Never raises. Surface the paths in the response.
271         paths = emit_indexer_report(
272             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
273             val_results=val_results, val_context=val_context,
274             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
275             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
276         )
277         if paths and isinstance(response, dict):
278             response["reports"] = paths
279
280
281 @app.post("/api/process", status_code=202)
282 def process_video(req: ProcessRequest, request: Request):
283     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{{job_id}} for state.
284

```

```

285     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
286     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
287     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
288     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
289     ""
290     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
291     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
292     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
293     try:
294         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295     except edge_auth.EdgeAuthError as e:
296         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
297
298     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299     try:
300         validate_input_path(req.video_path, allowed_root=allowed_root)
301         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
304     except ValueError as e:
305         raise HTTPException(status_code=400, detail=str(e)) from e
306     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
307     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
308     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
309     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
310     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
311     # rejects a non-local URI as out-of-root.
312     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
313         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
314     if req.segmenter and not segmenter_registry.get(req.segmenter):
315         available = list(segmenter_registry.list_available().keys())
316         raise HTTPException(
317             status_code=400,
318             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
319         )
320
321     job_id = uuid.uuid4().hex
322     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
323                                req.player_pool, req.max_frames, owner_id=owner_id):
324         raise HTTPException(status_code=500, detail="Failed to persist job record.")
325     _get_executor().submit(_drain_due_jobs)
326     return JSONResponse(
327         status_code=202,
328         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
329     )
330
331
332     @app.get("/api/jobs/{job_id}/cost")
333     def get_job_cost(job_id: str):
334         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
335         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.

```

```

`cost_usd` is
336     null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
337     cost = db_instance.get_job_cost(job_id)
338     if cost is None:
339         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
340     fx = global_config.billing.fx_inr_per_usd
341     usd = cost.get("cost_usd")
342     cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
343     cost["fx_inr_per_usd"] = fx
344     return cost
345
346
347     @app.get("/api/videos/{video_id}/cost")
348     def get_video_cost(video_id: str):
349         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
350         ``total_cost_inr`` + the
351         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
352         several."""
353         agg = db_instance.get_video_cost(video_id)
354         fx = global_config.billing.fx_inr_per_usd
355         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
356         agg["fx_inr_per_usd"] = fx
357         return agg
358
359     @app.get("/api/jobs/{job_id}")
360     def get_job(job_id: str):
361         """Job state: {status, progress, result, reports}. `status` = execution state
362         (queued|running|done|failed); the pipeline verdict is result["status"]."""
363         job = db_instance.get_job(job_id)
364         if not job:
365             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
366         result = job.get("result")
367         return {
368             "job_id": job["id"],
369             "status": job["status"],
370             "progress": job.get("progress"),
371             "video_id": job.get("video_id"),
372             "error": job.get("error"),
373             "result": result,
374             "reports": (result or {}).get("reports"),
375             "created_at": job.get("created_at"),
376             "started_at": job.get("started_at"),
377             "finished_at": job.get("finished_at"),
378         }
379
380     # --- Chunked upload (B2, PR-2) -----
381     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
382     # in
383     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
384     # via
385     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
386     # sequential-part logic, and abort cleanup ? is unchanged.
387     from backend.api import uploads as uploads_api
388
389     app.include_router(uploads_api.router)
390
391     # --- Highlight download (B3, PR-2) -----
392
393     @app.get("/api/highlights/{video_id}/{filename}")
394     def download_highlight(video_id: str, filename: str):
395         """Stream a compiled highlight reel ? `filename` is the bare name given to
396         /api/compile
397         (which only writes into output_dir; the browser previously got back an unreachable

```



```

395     server-local path).""
396     try:
397         name = safe_output_filename(filename)
398     except ValueError as e:
399         raise HTTPException(status_code=400, detail=str(e)) from e
400     if not db_instance.get_video(video_id):
401         raise HTTPException(status_code=404, detail="Indexed video record not found.")
402     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
403     or "output"
404     path = os.path.join(output_dir, name)
405     try:
406         path = resolve_under_root(path, output_dir)
407     except ValueError as e:
408         raise HTTPException(status_code=400, detail=str(e)) from e
409     if not os.path.isfile(path):
410         raise HTTPException(status_code=404,
411                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
412     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
413     return FileResponse(path, media_type=media, filename=name)
414
415 @app.post("/api/query")
416 def query_play_segments(req: QueryRequest):
417     filters = {
418         "server": req.server,
419         "receiver": req.receiver,
420         "duration_min": req.duration_min,
421         "event_type": req.event_type
422     }
423     segments = db_instance.query_play_segments(req.video_id, filters)
424     return {"play_segments": segments}
425
426 @app.post("/api/compile")
427 def compile_highlights(req: CompileRequest):
428     video = db_instance.get_video(req.video_id)
429     if not video:
430         raise HTTPException(status_code=404, detail="Indexed video record not found.")
431
432     # Retrieve matching play segments from db
433     all_segments = db_instance.query_play_segments(req.video_id, {})
434     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
435
436     if not matched_segments:
437         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
438
439     # Reject path traversal / absolute output names (os.path.join would otherwise let an
440     # absolute or '..'-laden output_filename write anywhere on disk).
441     try:
442         out_name = safe_output_filename(req.output_filename)
443     except ValueError as e:
444         raise HTTPException(status_code=400, detail=str(e)) from e
445
446     # The configured shot-count floor (stitching.min_shot_count) applies on the API
447     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
448     # metadata are exempt: the floor filters known counts only, so explicitly
449     # requested manual/legacy segments can't silently vanish under the default floor.
450     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
451     kept = []
452     for s in matched_segments:
453         meta = s.get("metadata") or {}
454         sc = meta.get("shot_count") if isinstance(meta, dict) else None
455         try:

```

```

456         if sc is not None and int(sc) < stitching.min_shot_count:
457             continue
458         except (TypeError, ValueError):
459             pass # unparseable count -> treat as unknown, keep
460         kept.append(s)
461     if len(kept) < len(matched_segments):
462         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
463                 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
464     if not kept:
465         raise HTTPException(status_code=400,
466                             detail=f"All {len(matched_segments)} matching segments fall
below "
467                                 f"stitching.min_shot_count={stitching.min_shot_count}.")
468
469     # Extract start/end time pairs
470     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
471
472     # Run compiler with the configured stitching paddings + optional branding watermark
473     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
474     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
475                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
476     try:
477         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
478         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
479         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
480         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
481         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
482         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
483         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
484         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
485         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
486         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
487         return {"status": "success", "filepath": out_path, "download_url": download_url}
488     except Exception as e:
489         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
490
491     # --- CLI Debug Utility & Orchestration ---
492     def run_cli_diagnose_clip(video_path: str, timestamp: float):
493         """CLI debugging utility to examine segment properties around a timestamp."""
494         print("=" * 60)
495         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
496         print("=" * 60)
497
498         cfg = load_config() # legacy wrapper still works, returns dict for now
499
500         # 1. Validation check diagnostics
501         print("[DIAGNOSTIC] Running validation framework...")
502         results = registry.run_all(video_path, cfg)
503         for r in results:
504             status_symbol = "[PASS]" if r.passed else "[FAIL]"
505             print(f"    {status_symbol} {r.validator_name}: {r.message}")

```

```

506         if r.details:
507             print(f"         Details: {r.details}")
508
509     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
510     print("-" * 60)
511     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
512     import cv2
513     cap = cv2.VideoCapture(video_path)
514     if not cap.isOpened():
515         print("[FAIL] OpenCV could not open video.")
516         return
517
518     fps = cap.get(cv2.CAP_PROP_FPS)
519     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
520
521     start_frame = max(0, int((timestamp - 3.0) * fps))
522     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
523
524     # Measure frame-by-frame changes in sample region
525     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
526     prev_gray = None
527     motions = []
528
529     for _ in range(start_frame, end_frame):
530         ret, frame = cap.read()
531         if not ret:
532             break
533         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
534         gray = cv2.GaussianBlur(gray, (21, 21), 0)
535
536         if prev_gray is not None:
537             diff = cv2.absdiff(prev_gray, gray)
538             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
539             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
540             motions.append(motion_ratio)
541             prev_gray = gray
542
543     cap.release()
544
545     if motions:
546         avg_motion = sum(motions) / len(motions)
547         # Support both legacy dict (from wrapper) and future direct model
548         if hasattr(cfg, "indexing"):
549             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
550         else:
551             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
552         print(f"    Sample Frame Count: {len(motions)}")
553         print(f"    Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
554         if avg_motion > threshold:
555             print("    Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
556         else:
557             print("    Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
558     else:
559         print("    [ERROR] No visual frames were extracted in the sample range.")
560
561     print("=" * 60)
562
563
564     def _enforce_startup_or_exit(config) -> None:
565         """M5: run the per-profile startup checks for the server/CLI path and

```

```

``sys.exit(1)`` on a
566     FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
any work.
567     A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No
GPU probe
568     here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
569     so the fail-fast wiring + the default-OFF no-op are unit-testable.""
570     try:
571         enforce_startup_checks(config, logger=logger)
572     except StartupCheckError as e:
573         logger.error("[STARTUP] refusing to start: %s", e)
574         sys.exit(1)
575
576
577     def main():
578         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
579         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
580         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
581         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
582         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
583         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
584         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
585         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
586
587         available_segmenters = list(segmenter_registry.list_available().keys())
588         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
589         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
590         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
591         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
592
593         args = parser.parse_args()
594
595         if args.setup:
596             # Invoke the diagnostics/bootstrap script (renamed from the old root
597             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
598             # build system). Resolve its path from this file so `indexer --setup`
599             # works regardless of the caller's cwd.
600             import subprocess
601             doctor_path = os.path.join(
602                 os.path.dirname(os.path.abspath(__file__)),
603                 "scripts", "doctor.py",
604             )
605             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
606             subprocess.run([sys.executable, doctor_path])
607             return
608
609         if args.trim_start is not None and args.trim_end is not None:
610             if not args.video_path:
611                 print("[ERROR] Please provide --video-path to extract a segment.")
612                 return
613
614             # Determine output path
615             out_filename = args.trim_output or "trimmed_segment.mp4"

```

```

616         if os.path.isabs(out_filename):
617             out_path = out_filename
618             out_dir = os.path.dirname(out_path)
619             out_name = os.path.basename(out_path)
620         else:
621             out_dir = args.output_dir
622             out_path = os.path.join(out_dir, out_filename)
623             out_name = out_filename
624
625         os.makedirs(out_dir, exist_ok=True)
626         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
627
628         # global_config isn't initialized this early; load the typed config so the trim
629         # clip carries the same configured stitching paddings + watermark as a real
630         # highlight (watermark default-on).
631         stitching = load_app_config().stitching
632         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
633                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
634         try:
635             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
636             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
637         except Exception as e:
638             print(f"[ERROR] Failed to compile trimmed segment: {e}")
639         return
640
641     if args.debug_clip is not None:
642         if not args.video_path:
643             print("[ERROR] Please provide --video-path to debug a specific clip.")
644             return
645         run_cli_diagnose_clip(args.video_path, args.debug_clip)
646         return
647
648     # Initialize Global Config and DB
649     global global_config, db_instance
650     global_config = load_app_config() # returns typed AppConfig
651     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
652
653     # The CLI --output-dir overrides the configured output directory. It now lives
654     # on the typed model (OutputConfig.output_dir), so every consumer ? including
655     # /api/compile ? reads the same value instead of a write-only runtime hack.
656     global_config.output.output_dir = args.output_dir
657     os.makedirs(global_config.output.output_dir, exist_ok=True)
658
659     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
660     db_instance = Database(db_path)
661
662     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
663     # the executor is in-process. Mark them failed so pollers see a terminal state.
664     swept = db_instance.fail_stale_jobs()
665     if swept:
666         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
667
668     # CLI processing mode (no web UI needed)
669     if args.video_path and not args.server:
670         print(f"[CLI] Processing video file: {args.video_path}")
671         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
        # fetched to scratch.
672         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
        # key, not the

```

```

673         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
674         storage_cfg = getattr(global_config, "storage", None)
675         try:
676             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
677         except ValueError as e:
678             print(f"[FAIL] {e}")
679             return
680         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
681             print(f"[FAIL] Video file not found: {args.video_path}")
682             return
683         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
684
685         video_id = compute_video_id(local_video)
686         was_reingest = db_instance.get_video(video_id) is not None
687
688         # Validate (with-context variant surfaces ffprobe_meta for the report)
689         print("[CLI] Validating...")
690         val_results, val_context = registry.run_all_with_context(local_video,
global_config)
691         failures = [r for r in val_results if not r.passed]
692
693         # Save video status
694         status = "failed" if failures else "processed"
695         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
696
697         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
698         segmenter_actual = None
699         segmentation_ran = False
700         try:
701             print("\n" + "="*40)
702             print("VALIDATION SUMMARY")
703             print("="*40)
704             for r in val_results:
705                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
706                 print(f"{status_symbol} {r.validator_name}: {r.message}")
707             print("="*40 + "\n")
708
709             if failures:
710                 print("[FAIL] Video failed checks. Indexing halted.")
711                 return
712
713             # Index
714             segmenter_cls = segmenter_registry.get(seg_name)
715             if not segmenter_cls:
716                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
717                 return
718
719             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
720             segmenter_actual = seg_name
721             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
722             segmenter = segmenter_cls(db_instance, global_config)
723             result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
724             segmentation_ran = True
725
726             if isinstance(result, Failure):
727                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
728                 print("[FAIL] Indexing halted for video.")
729                 segments = []
730             else:
731                 segments = result.value if hasattr(result, "value") else result
732                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play

```

```

segments inside database: {db_path}")
733         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
734         _finalize_reingest(video_id, segments, play_wm, cand_wm)
735     finally:
736         # Always emit the per-video observability report. Never raises.
737         paths = emit_indexer_report(
738             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
739             val_results=val_results, val_context=val_context,
740             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
741             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
742         )
743     if paths:
744         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
745         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
746         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
747     return
748
749     # Start Local Server Mode
750     if args.server or not args.video_path:
751         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
752         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
753         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
754         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
755         _get_executor().submit(_drain_due_jobs)
756         uvicorn.run(app, host="127.0.0.1", port=args.port)
757
758     if __name__ == "__main__":
759         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
760         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
761         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
762         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
763         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
764         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
765         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
766         import backend.main as _bm
767
768         _bm.main()
769

```

8. assistant (2026-06-21T14:36:55.966Z)

Excellent. Now let me search for the edge_auth module and CORS-related code:

9. user (2026-06-21T14:36:56.932Z)

```

1     """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3     **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4     ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5     ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6     signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't

```

```

spooft** the
7     identity header (the explicit D9 anti-spoof requirement). Returns the verified
  ``user_id`` (the
8     token's ``email``, else ``sub``).
9
10    ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11    ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12    unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13    """
14
15    from __future__ import annotations
16
17    import json
18    import time
19    import urllib.request
20    from typing import Any, Callable, Mapping, Optional
21
22    # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23    CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24    _JWKS_PATH = "/cdn-cgi/access/certs"
25    _JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27    JwksFetcher = Callable[[str], dict]
28
29
30    class EdgeAuthError(Exception):
31        """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34    # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35    _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38    def _issuer(team_domain: str) -> str:
39        return team_domain.rstrip("/")
40
41
42    def _default_jwks_fetcher(team_domain: str) -> dict:
43        """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the
44        production path reaches here; tests pass an injected fetcher."""
45        url = _issuer(team_domain) + _JWKS_PATH
46        with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
47            data: Any = json.load(resp)
48            if not isinstance(data, dict) or "keys" not in data:
49                raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50            return data
51
52
53    def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54        """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
55        bypasses the cache so each test is deterministic."""
56        if fetcher is not None:
57            return fetcher(team_domain)
58        now = time.time()
59        cached = _jwks_cache.get(team_domain)

```



```

60         if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61             return cached[1]
62         jwks = _default_jwks_fetcher(team_domain)
63         _jwks_cache[team_domain] = (now, jwks)
64         return jwks
65
66
67     def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:
68         """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
69         return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
70         try:
71             import jwt
72             from jwt import PyJWKSet
73         except Exception as exc: # pragma: no cover - only when the auth extra isn't
installed
74             raise EdgeAuthError(
75                 "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76                 "(pip install -e .[auth]) on a host with edge_auth_enabled."
77             ) from exc
78
79         try:
80             kid = jwt.get_unverified_header(token).get("kid")
81             if not kid:
82                 raise EdgeAuthError("token header has no kid")
83             signing_key = PyJWKSet.from_dict(jwks)[kid]
84             claims = jwt.decode(
85                 token, signing_key.key, algorithms=["RS256"],
86                 audience=aud, issuer=_issuer(team_domain),
87                 options={"require": ["exp", "iss", "aud"]},
88             )
89         except EdgeAuthError:
90             raise
91         except KeyError as exc: # kid not in the JWKS
92             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93         except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94             raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96         user = claims.get("email") or claims.get("sub")
97         if not user:
98             raise EdgeAuthError("verified token carries no email/sub claim")
99         return str(user)
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                       jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
user; unchanged).
107         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
on any miss.
108         ``jwks_fetcher`` is injectable for offline tests.
109         """
110         sec = getattr(config, "security", None)
111         if not sec or not getattr(sec, "edge_auth_enabled", False):
112             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
113
114         team_domain = getattr(sec, "cf_team_domain", "") or ""
115         aud = getattr(sec, "cf_access_aud", "") or ""
116         if not team_domain or not aud:
117             raise EdgeAuthError(

```

```

118         "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
configured")
119
120     # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
support a plain
121     # dict too (tests) by trying both casings.
122     token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
123     if not token:
124         raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
Cloudflare Access)")
125
126     jwks = _get_jwks(team_domain, jwks_fetcher)
127     return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
128

```

10. user (2026-06-21T14:36:58.133Z)

```

backend/api/auth.py: """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant
``user_id`` (M9, D9).
backend/api/auth.py: **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver
is a strict NO-OP (returns
backend/api/auth.py: ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256)
against the team JWKS ?
backend/api/auth.py: CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
backend/api/auth.py:     """A required Cf-Access token was missing or failed verification ? the
caller returns 401/403."""
backend/api/auth.py:     """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp +
issuer + audience) and
backend/api/auth.py:         "(pip install -e .[auth]) on a host with edge_auth_enabled."
backend/api/auth.py:         raise EdgeAuthError(f"Cf-Access token verification failed: {exc}")
from exc
backend/api/auth.py:     DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local
single-user; unchanged).
backend/api/auth.py:     Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError``
(? 401/403) on any miss.
backend/api/auth.py:     if not sec or not getattr(sec, "edge_auth_enabled", False):
backend/api/auth.py:         "edge_auth_enabled but security.cf_team_domain / cf_access_aud
are not configured")
backend/api/job_worker.py:     claimed row (carries ``owner_id``); ``result`` is the pipeline
output."""
backend/api/job_worker.py:         owner_id=job.get("owner_id"), margin=billing_cfg.margin,
backend/config/models.py:     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is
required/verified (the local single-user
backend/config/models.py:     # path is unchanged). When True, /api/process requires + verifies
the `Cf-Access-Jwt-Assertion`
backend/config/models.py:     # header) and tags the job with the verified user_id (owner_id).
The team domain (e.g.
backend/config/models.py:     edge_auth_enabled: bool = False
backend/config/models.py:     # OFF ? Cf-Access is a header, not a cookie.)
backend/infrastructure/database.py:         ("owner_id", "TEXT"),           # who pays
(the non-owner user / tenant)
backend/infrastructure/database.py:         cursor.execute("CREATE INDEX IF NOT EXISTS
idx_jobs_owner ON jobs(owner_id) "
backend/infrastructure/database.py:             "WHERE owner_id IS NOT NULL")
backend/infrastructure/database.py:             owner_id: Optional[str] = None) -> bool:
backend/infrastructure/database.py:         (EXECUTION_POLICY.md) the worker honours; None ? the
worker's default policy. ``owner_id``
backend/infrastructure/database.py:         "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?,
?),",
backend/infrastructure/database.py:         max_frames, json.dumps(policy) if policy is not
None else None, owner_id),
backend/infrastructure/database.py:             owner_id: Optional[str] = None,
margin: float = 0.0) -> Optional[Dict[str, Any]]:
backend/infrastructure/database.py:         "UPDATE jobs SET owner_id=?, compute_backend=?,
gpu_type=?, gpu_active_seconds=?, "

```

```

backend/infrastructure/database.py:          (owner_id, compute_backend, gpu_type,
usage.get(billing.GPU_SECONDS),
backend/infrastructure/database.py:          "SELECT id, video_id, owner_id,
compute_backend, gpu_type, gpu_active_seconds, "
backend/main.py:from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth
(default-OFF)
backend/main.py: # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the
local single-user,
backend/main.py: # unchanged). When edge_auth_enabled, the Cf-Access JWT is required +
verified (a missing or
backend/main.py: # spoofed identity header fails here with 401, before any work) and tags the
job's owner_id.
backend/main.py:          owner_id = edge_auth.resolve_tenant(request.headers, global_config)
backend/main.py:          except edge_auth.EdgeAuthError as e:
backend/main.py:          req.player_pool, req.max_frames,
owner_id=owner_id):

```

11. assistant (2026-06-21T14:37:00.182Z)

Now let me check the config models to understand the security configuration:

12. user (2026-06-21T14:37:01.654Z)

```

class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)

class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    relevance: ValidationRelevanceConfig = Field(default_factory=ValidationRelevanceConfig)

class TrackNetConfig(BaseModel):
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""
    queue_length: int = 5
    stage_in_wsl: bool = True
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
    # decoded frame cache) are deleted automatically ? they are pure intermediates,
    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
    keep_frames: bool = False
    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
    # 0 = disabled.
    min_free_gb: float = 0.0

class WasbIndexConfig(BaseModel):
    """Typed config for the live WASB shuttle substrate (indexing.wasb).

    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
    these knobs actually take effect ? previously this whole block was silently dropped
    because nested config sections defaulted to extra='ignore'.

```

```

"""
wsl_distro: str = "Ubuntu"
repo_dir: str = "~/models/WASB-SBDT"
conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
conda_env: str = "wasb"
weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
sport: str = "badminton"
wsl_stage_dir: str = "~/clips"
timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
velocity_threshold: float = 0.02
# --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
# Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
# torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
fallback
# for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
downgrade).
device: str = "cuda" # auto | cuda | mps | cpu
--
class FusionConfig(BaseModel):
    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

    Every default reproduces control behaviour: the fusion segmenter persists the
    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
    true ? the person-cue features, but it does NOT gate the served handoff unless a
    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
    supplied ? off-court persons (spectators / adjacent courts) are excluded.
    """
    # Which trajectory substrate seeds the windows (shuttle stays primary).
    substrate: Literal["wasb", "tracknet"] = "wasb"
    # Windowing velocity threshold for the fusion family (the engine reads
    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
    velocity_threshold: float = 0.02
    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
    # enabled ? so the default path never imports torch / touches the GPU.
    cue_flags: dict[str, bool] = Field(default_factory=dict)
    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
    scoring).
    --

```

13. assistant (2026-06-21T14:37:04.194Z)

Let me get the SecurityConfig class specifically:

14. user (2026-06-21T14:37:04.949Z)

```

# buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
# them to WARNING; set true to restore full chatter for request-flow debugging.
verbose_http: bool = False

```

```

class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).

```

```

max_upload_mb: int = 4096
max_part_mb: int = 95
allowed_upload_extensions: List[str] = ["mp4", "mov"]

# --- M9 edge auth (Cloudflare Access, D9) ---
# DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
# path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
# header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
# header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
# ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
# application's audience tag. NO secrets here ? these are public identifiers.
edge_auth_enabled: bool = False
cf_team_domain: str = ""          # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
JWKS)
cf_access_aud: str = ""          # the CF Access application AUD tag the token must carry
# (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
# backend/main.py so importing the app does no filesystem I/O; default ["*"]. Credentials
stay
# OFF ? Cf-Access is a header, not a cookie.)

```

15. assistant (2026-06-21T14:37:08.049Z)

Perfect. Now let me check the SPA in the khelsutra project to understand how it calls the API:

16. user (2026-06-21T14:37:09.544Z)

```

./site/node_modules/@asamuzakjp/nwsapi/src/nwsapi.js
./site/node_modules/@exodus/bytes/fallback/encoding.api.js
./site/node_modules/jsdom/lib/api.js
./site/node_modules/jsdom/lib/jsdom/living/file-api/Blob-impl.js
./site/node_modules/jsdom/lib/jsdom/living/file-api/File-impl.js
./site/node_modules/jsdom/lib/jsdom/living/file-api/FileList-impl.js
./site/node_modules/jsdom/lib/jsdom/living/file-api/FileReader-impl.js
./site/node_modules/mdn-data/api/index.js
./site/node_modules/undici/lib/api/abort-signal.js
./site/node_modules/undici/lib/api/api-connect.js
./site/node_modules/undici/lib/api/api-pipeline.js
./site/node_modules/undici/lib/api/api-request.js
./site/node_modules/undici/lib/api/api-stream.js
./site/node_modules/undici/lib/api/api-upgrade.js
./site/node_modules/undici/lib/api/index.js

```

17. assistant (2026-06-21T14:37:11.390Z)

Let me look for source files excluding node_modules:

18. user (2026-06-21T14:37:12.087Z)

(Bash completed with no output)

19. user (2026-06-21T14:37:14.477Z)

```

total 81
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:12 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 ..
-rw-r--r-- 1 avidu 197609  1757 Jun 20 11:30 README.md
-rw-r--r-- 1 avidu 197609 12288 Jun 20 20:15 access.db
-rw-r--r-- 1 avidu 197609  2369 Jun 20 11:30 make_og.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 08:16 node_modules
-rw-r--r-- 1 avidu 197609 19438 Jun 21 08:20 package-lock.json
-rw-r--r-- 1 avidu 197609   434 Jun 21 08:20 package.json

```

```
drwxr-xr-x 1 avidu 197609      0 Jun 21 13:08 public
-rw-r--r-- 1 avidu 197609    849 Jun 20 11:30 schema.sql
drwxr-xr-x 1 avidu 197609      0 Jun 20 19:49 scripts
-rw-r--r-- 1 avidu 197609   3406 Jun 21 12:12 server.js
drwxr-xr-x 1 avidu 197609      0 Jun 21 13:08 src
-rw-r--r-- 1 avidu 197609   1064 Jun 20 12:52 wrangler.toml
```

20. user (2026-06-21T14:37:16.873Z)

```
site/src/assets.test.js
site/src/deploy-config.test.js
site/src/index.js
site/src/index.test.js
site/src/notify.js
site/src/server.test.js
site/src/store.js
site/src/ux.dom.test.js
```

21. assistant (2026-06-21T14:37:19.269Z)

This is a different kind of project. Let me check if there's a frontend SPA elsewhere. First, let me look at the badminton-highlight-indexer backend/api/static directory:

22. user (2026-06-21T14:37:19.915Z)

```
backend/api/auth.py
backend/api/job_worker.py
backend/api/static/app.js
backend/api/static/index.html
backend/api/static/style.css
backend/api/uploads.py
backend/api/__init__.py
backend/api/__pycache__/auth.cpython-313.pyc
backend/api/__pycache__/job_worker.cpython-313.pyc
backend/api/__pycache__/uploads.cpython-313.pyc
backend/api/__pycache__/__init__.cpython-313.pyc
```

23. assistant (2026-06-21T14:37:21.908Z)

Excellent! Let me read the SPA client code:

24. user (2026-06-21T14:37:22.549Z)

```
1      // Badminton Highlight & Rally Indexer JS Client
2
3      document.addEventListener("DOMContentLoaded", () => {
4          // --- APP STATE ---
5          let currentVideoId = null;
6          let indexedRallies = [];
7          let selectedRallyIds = new Set();
8          let playerPool = ["Avi", "Suresh", "Ramesh", "Deepak"];
9
10         // --- UI ELEMENTS ---
11         const videoPathInput = document.getElementById("video-path-input");
12         const playerPoolInput = document.getElementById("player-pool-input");
13         const btnProcess = document.getElementById("btn-process-video");
14
15         const validationList = document.getElementById("validation-checker-list");
16         const ralliesResultsList = document.getElementById("rallies-results-list");
17         const rallyCountLabel = document.getElementById("rally-count-label");
18
19         // Query Filters
20         const queryServer = document.getElementById("query-server");
21         const queryReceiver = document.getElementById("query-receiver");
```

```

22     const queryDuration = document.getElementById("query-duration");
23     const queryEvent = document.getElementById("query-event");
24     const btnQuery = document.getElementById("btn-run-query");
25
26     // Compiler Bar
27     const compileBoard = document.getElementById("highlight-compile-board");
28     const compileSelectedCount = document.getElementById("compile-selected-count");
29     const highlightOutputName = document.getElementById("highlight-output-name");
30     const btnCompile = document.getElementById("btn-compile-highlights");
31
32     // Loading Overlay
33     const loadingOverlay = document.getElementById("loading-overlay");
34     const loadingTitle = document.getElementById("loading-title");
35     const loadingSubtitle = document.getElementById("loading-subtitle");
36
37     // --- HELPERS ---
38     const showLoader = (title, subtitle) => {
39         loadingTitle.textContent = title;
40         loadingSubtitle.textContent = subtitle;
41         loadingOverlay.style.display = "flex";
42     };
43
44     const hideLoader = () => {
45         loadingOverlay.style.display = "none";
46     };
47
48     const populatePlayerSelects = (players) => {
49         // Clear except first
50         queryServer.innerHTML = '<option value="">Any Server</option>';
51         queryReceiver.innerHTML = '<option value="">Any Receiver</option>';
52
53         players.forEach(player => {
54             const opt1 = document.createElement("option");
55             opt1.value = player;
56             opt1.textContent = player;
57             queryServer.appendChild(opt1);
58
59             const opt2 = document.createElement("option");
60             opt2.value = player;
61             opt2.textContent = player;
62             queryReceiver.appendChild(opt2);
63         });
64     };
65
66     // Initialize player selects at start
67     populatePlayerSelects(playerPool);
68
69     // --- 1. RUN VIDEO PROCESSING & VALIDATIONS ---
70     btnProcess.addEventListener("click", async () => {
71         const videoPath = videoPathInput.value.trim();
72         if (!videoPath) {
73             alert("Please enter a valid local MP4 video file path.");
74             return;
75         }
76
77         // Parse custom players
78         const rawPlayers = playerPoolInput.value.trim();
79         if (rawPlayers) {
80             playerPool = rawPlayers.split(",").map(p => p.trim()).filter(p => p.length >
0);
81             populatePlayerSelects(playerPool);
82         }
83
84         showLoader("Analyzing Video Stream", "Running structural diagnostics, video
focus checks, and relevance validators...");

```

```

85
86     try {
87         // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
88         const response = await fetch("/api/process", {
89             method: "POST",
90             headers: { "Content-Type": "application/json" },
91             body: JSON.stringify({
92                 video_path: videoPath,
93                 player_pool: playerPool
94             })
95         });
96
97         if (!response.ok) {
98             const err = await response.json();
99             throw new Error(err.detail || "Server failed to process the request.");
100         }
101
102         const submitted = await response.json();
103         const jobId = submitted.job_id;
104
105         // Poll until the job reaches a terminal state (done | failed).
106         let job;
107         for (;;) {
108             const jr = await fetch(`/api/jobs/${jobId}`);
109             if (!jr.ok) {
110                 const err = await jr.json();
111                 throw new Error(err.detail || "Failed to poll job state.");
112             }
113             job = await jr.json();
114             if (job.status === "done" || job.status === "failed") break;
115             showLoader("Analyzing Video Stream",
116                 `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
117             await new Promise(res => setTimeout(res, 3000));
118         }
119
120         if (job.status === "failed") {
121             throw new Error(job.error || "Processing job failed.");
122         }
123
124         const data = job.result || {};
125         currentVideoId = data.video_id;
126
127         // Render validation list
128         renderValidationResults(data.results);
129
130         if (data.status === "failed") {
131             hideLoader();
132             alert("Validation Failed! This video does not meet our quality or
structural criteria. Please review the checklist.");
133             renderEmptyRallies("Validation failed. Please load a valid badminton raw
video.");
134             compileBoard.style.display = "none";
135             return;
136         }
137
138         // Successfully processed (field is play_segments ? the old `rallies` read
was a bug)
139         indexedRallies = data.play_segments || [];
140         selectedRallyIds.clear();
141         renderRallyCards(indexedRallies);
142
143         hideLoader();
144         alert(`Success! Successfully processed and indexed ${indexedRallies.length}
badminton rallies.`);
145

```



```

146     } catch (error) {
147         hideLoader();
148         alert(`Error: ${error.message}`);
149         renderEmptyRallies("Error processing file.");
150     }
151 });
152
153 // --- 2. RENDER VALIDATION CHECKLIST ---
154 const renderValidationResults = (results) => {
155     validationList.innerHTML = "";
156
157     results.forEach(res => {
158         const div = document.createElement("div");
159         div.className = `validation-item ${res.passed ? 'passed' : 'failed'}`;
160
161         const icon = document.createElement("div");
162         icon.className = "val-icon";
163         icon.textContent = res.passed ? "?" : "?";
164
165         const info = document.createElement("div");
166         info.className = "val-info";
167
168         const title = document.createElement("h4");
169         title.textContent = formatValidatorName(res.validator_name);
170
171         const desc = document.createElement("p");
172         desc.textContent = res.message;
173
174         info.appendChild(title);
175         info.appendChild(desc);
176
177         // If there are detailed metrics, render them
178         if (res.details && Object.keys(res.details).length > 0) {
179             const detailsSpan = document.createElement("span");
180             detailsSpan.className = "val-details";
181             detailsSpan.textContent = formatDetails(res.details);
182             info.appendChild(detailsSpan);
183         }
184
185         div.appendChild(icon);
186         div.appendChild(info);
187         validationList.appendChild(div);
188     });
189 };
190
191 const formatValidatorName = (name) => {
192     return name.split("_").map(w => w.charAt(0).toUpperCase() + w.slice(1)).join("
");
193 };
194
195 const formatDetails = (details) => {
196     return Object.entries(details)
197         .map([k, v] => `${k}: ${v}`)
198         .join(" | ");
199 };
200
201 // --- 3. RENDER RALLY TIMELINE CARDS ---
202 const renderRallyCards = (rallies) => {
203     ralliesResultsList.innerHTML = "";
204     selectedRallyIds.clear();
205     updateCompilerBar();
206
207     rallyCountLabel.textContent = `${rallies.length} Rallies Found`;
208
209     if (!rallies || rallies.length === 0) {

```

```

210         renderEmptyRallies("No rallies found matching your filters.");
211         return;
212     }
213
214     rallies.forEach(rally => {
215         const card = document.createElement("div");
216         card.className = `rally-card ${selectedRallyIds.has(rally.id) ? 'selected' :
217         ''}`;
218
219         card.setAttribute("id", `rally-card-${rally.id}`);
220
221         const meta = document.createElement("div");
222         meta.className = "rally-meta";
223
224         const title = document.createElement("h3");
225         title.textContent = `Rally #${rally.id} (Serve: ${rally.server} ? Receive:
226         ${rally.receiver})`;
227
228         const sub = document.createElement("p");
229         sub.innerHTML = `
230             <span><strong>Start:</strong> ${formatTime(rally.start_time)}</span>
231             <span><strong>End:</strong> ${formatTime(rally.end_time)}</span>
232         `;
233
234         const tagsDiv = document.createElement("div");
235         tagsDiv.className = "rally-tags";
236
237         // Long rally tag
238         if (rally.duration > 10) {
239             const longTag = document.createElement("span");
240             longTag.className = "rally-tag long";
241             longTag.textContent = "Long Rally";
242             tagsDiv.appendChild(longTag);
243         }
244
245         // Check events to see if we have smashes or fouls
246         const events = rally.events || [];
247         const hasSmash = events.some(e => e.type === "smash");
248         const hasFoul = events.some(e => e.type === "foul");
249
250         if (hasSmash) {
251             const smashTag = document.createElement("span");
252             smashTag.className = "rally-tag smash";
253             smashTag.textContent = "Contains Smash";
254             tagsDiv.appendChild(smashTag);
255         }
256
257         if (hasFoul) {
258             const foulTag = document.createElement("span");
259             foulTag.className = "rally-tag foul";
260             foulTag.textContent = "Ends in Foul";
261             tagsDiv.appendChild(foulTag);
262         }
263
264         meta.appendChild(title);
265         meta.appendChild(sub);
266         meta.appendChild(tagsDiv);
267
268         const action = document.createElement("div");
269         action.className = "rally-action";
270
271         const dur = document.createElement("div");
272         dur.className = "rally-dur";
273         dur.textContent = `${rally.duration.toFixed(1)}s`;
274
275         const chk = document.createElement("div");

```

```

273         chk.className = "checkbox-custom";
274
275         action.appendChild(dur);
276         action.appendChild(chk);
277
278         card.appendChild(meta);
279         card.appendChild(action);
280
281         // Card Toggle Selection Click Listener
282         card.addEventListener("click", () => {
283             if (selectedRallyIds.has(rally.id)) {
284                 selectedRallyIds.delete(rally.id);
285                 card.classList.remove("selected");
286             } else {
287                 selectedRallyIds.add(rally.id);
288                 card.classList.add("selected");
289             }
290             updateCompilerBar();
291         });
292
293         ralliesResultsList.appendChild(card);
294     });
295 };
296
297 const renderEmptyRallies = (msg) => {
298     ralliesResultsList.innerHTML = `
299         <div class="empty-state">
300             <svg width="48" height="48" fill="none" stroke="currentColor" stroke-
width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M3.75
5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
301             <h3>No Rallies Available</h3>
302             <p>${msg}</p>
303         </div>
304     `;
305 };
306
307 const formatTime = (seconds) => {
308     const m = Math.floor(seconds / 60);
309     const s = Math.floor(seconds % 60);
310     const ms = Math.floor((seconds % 1) * 10);
311     return `${m.toString().padStart(2, '0')}:${s.toString().padStart(2,
'0')}.${ms}`;
312 };
313
314 // --- 4. RALLY TIMELINE FILTER QUERY ---
315 btnQuery.addEventListener("click", async () => {
316     if (!currentVideoId) {
317         alert("Please process a video first before running filters.");
318         return;
319     }
320
321     const server = queryServer.value;
322     const receiver = queryReceiver.value;
323     const duration = parseFloat(queryDuration.value) || null;
324     const event = queryEvent.value;
325
326     showLoader("Filtering Database Index", "Fetching rallies matching exact
relational conditions...");
327
328     try {
329         const res = await fetch("/api/query", {
330             method: "POST",
331             headers: { "Content-Type": "application/json" },
332             body: JSON.stringify({
333                 video_id: currentVideoId,

```

```

334         server: server || null,
335         receiver: receiver || null,
336         duration_min: duration || null,
337         event_type: event || null
338     });
339 });
340
341 if (!res.ok) throw new Error("Failed to filter index.");
342
343 const data = await res.json();
344 renderRallyCards(data.rallies || []);
345 } catch (err) {
346     alert(`Query failed: ${err.message}`);
347 } finally {
348     hideLoader();
349 }
350 });
351
352 // --- 5. COMPILER BOARD MANAGEMENT ---
353 const updateCompilerBar = () => {
354     const count = selectedRallyIds.size;
355     if (count > 0) {
356         compileBoard.style.display = "flex";
357         compileSelectedCount.textContent = `${count} Rally Segment${count > 1 ? 's'
: ''} Selected`;
358     } else {
359         compileBoard.style.display = "none";
360     }
361 };
362
363 btnCompile.addEventListener("click", async () => {
364     if (!currentVideoId || selectedRallyIds.size === 0) {
365         alert("No rallies selected for compiling highlights.");
366         return;
367     }
368
369     const outputName = highlightOutputName.value.trim() ||
"highlights_compiled.mp4";
370     showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
stitching demuxers...");
371
372     try {
373         const res = await fetch("/api/compile", {
374             method: "POST",
375             headers: { "Content-Type": "application/json" },
376             body: JSON.stringify({
377                 video_id: currentVideoId,
378                 // API field is segment_ids (the old rally_ids never matched the
request model)
379                 segment_ids: Array.from(selectedRallyIds),
380                 output_filename: outputName
381             })
382         });
383
384         if (!res.ok) {
385             const err = await res.json();
386             throw new Error(err.detail || "Concatenation failed.");
387         }
388
389         await res.json();
390         // B3 (PR-2): offer the reel via the streaming download endpoint instead of
391         // alerting a server-local path the browser can't reach.
392         const dlUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
393         let dl = document.getElementById("highlight-download-link");

```

```

394         if (!dl) {
395             dl = document.createElement("a");
396             dl.id = "highlight-download-link";
397             dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
398             compileBoard.appendChild(dl);
399         }
400         dl.href = dlUrl;
401         dl.setAttribute("download", outputName);
402         dl.textContent = `? Download ${outputName}`;
403         alert(`Highlight compilation completed! Use the download link to save
${outputName}.`);
404
405         } catch (error) {
406             alert(`Stitching compilation failed: ${error.message}`);
407         } finally {
408             hideLoader();
409         }
410     });
411 });
412

```

25. user (2026-06-21T14:37:22.958Z)

```

1      <!DOCTYPE html>
2      <html lang="en">
3      <head>
4          <meta charset="UTF-8">
5          <meta name="viewport" content="width=device-width, initial-scale=1.0">
6          <title>Badminton Highlight & Rally Indexer</title>
7          <link rel="stylesheet" href="/static/style.css">
8      </head>
9      <body>
10         <div class="app-container">
11             <!-- HEADER -->
12             <header>
13                 <div class="logo-section">
14                     <h1 id="app-title">Badminton Highlight & Rally Indexer</h1>
15                     <p>Local AI-powered video validation, segmentation, and clip
indexing</p>
16                 </div>
17                 <div class="status-badge">
18                     <span class="status-indicator"></span>
19                     <span>Local Server Active</span>
20                 </div>
21             </header>
22
23             <!-- MAIN GRID LAYOUT -->
24             <div class="main-grid">
25                 <!-- SIDEBAR: Controls & Validation -->
26                 <div class="sidebar">
27                     <!-- Video File Input Panel -->
28                     <div class="glass-panel" id="video-input-panel">
29                         <h2 class="panel-title">
30                             <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path
d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"/><circle
cx="12" cy="13" r="4"/></svg>
31                             Load Local Video
32                         </h2>
33                         <div class="form-group">
34                             <label for="video-path-input">Local Video File Path</label>
35                             <input type="text" id="video-path-input" class="text-input"
placeholder="e.g. c:/videos/badminton_gopro.mp4" value="">
36                         </div>
37                     </div>

```

```

38         <label for="player-pool-input">Player Mappings (Comma
Separated)</label>
39         <input type="text" id="player-pool-input" class="text-input"
placeholder="Avi, Suresh, Ramesh, Deepak" value="Avi, Suresh, Ramesh, Deepak">
40     </div>
41     <button id="btn-process-video" class="btn-primary">
42         <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polygon
points="5 3 19 12 5 21 5 3"/></svg>
43         Validate & Index Video
44     </button>
45 </div>
46
47 <!-- Validation Status Panel -->
48 <div class="glass-panel" id="validation-panel">
49     <h2 class="panel-title">
50         <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"/></svg>
points="22 4 12 14.01 9 11.01"/></svg>
51         Video Validation Checks
52     </h2>
53     <div class="validation-list" id="validation-checker-list">
54         <div class="empty-state" style="padding: 20px;">
55             <p style="font-size: 0.85rem;">Load a video to trigger the
pluggable validation checklist.</p>
56         </div>
57     </div>
58 </div>
59 </div>
60
61 <!-- WORKSPACE: Rallies Index Explorer -->
62 <div class="workspace">
63     <!-- Rallies timeline results panel -->
64     <div class="glass-panel rallies-explorer">
65         <div class="workspace-header">
66             <h2 class="panel-title" style="border: none; padding: 0;">
67                 <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><rect x="3" y="3" width="18" height="18" rx="2"
ry="2"/><line x1="9" y1="3" x2="9" y2="21"/><line x1="15" y1="3" x2="15" y2="21"/><line x1="3"
y1="9" x2="21" y2="9"/><line x1="3" y1="15" x2="21" y2="15"/></svg>
68                 Rally Database Index Explorer
69             </h2>
70             <span id="rally-count-label" class="rally-tag"
style="background: rgba(255,255,255,0.06);">0 Rallies Found</span>
71         </div>
72
73     <!-- Query Bar -->
74     <div class="query-bar">
75         <div class="form-group" style="gap: 4px;">
76             <label style="font-size: 0.75rem;">Server</label>
77             <select id="query-server">
78                 <option value="">Any Server</option>
79             </select>
80         </div>
81         <div class="form-group" style="gap: 4px;">
82             <label style="font-size: 0.75rem;">Receiver</label>
83             <select id="query-receiver">
84                 <option value="">Any Receiver</option>
85             </select>
86         </div>
87         <div class="form-group" style="gap: 4px;">
88             <label style="font-size: 0.75rem;">Min Duration</label>
89             <input type="number" id="query-duration" placeholder="e.g.
5" min="0" step="0.5">
90         </div>

```

```

91         <div class="form-group" style="gap: 4px;">
92             <label style="font-size: 0.75rem;">Event Filter</label>
93             <select id="query-event">
94                 <option value="">Any Events</option>
95                 <option value="smash">Smashes</option>
96                 <option value="serve">Serves Only</option>
97                 <option value="foul">Fouls Only</option>
98             </select>
99         </div>
100         <button id="btn-run-query" class="btn-secondary" style="height:
38px; margin-top: auto; padding: 0;">
101             Filter Index
102         </button>
103     </div>
104
105     <!-- Rally Cards Grid -->
106     <div class="rallies-grid" id="rallies-results-list">
107         <div class="empty-state">
108             <svg width="48" height="48" fill="none"
stroke="currentColor" stroke-width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round"
stroke-linejoin="round" d="M3.75 5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
109             <h3>Rally database is empty.</h3>
110             <p>Enter a local video filepath in the sidebar and run
indexing to build the database index.</p>
111         </div>
112     </div>
113 </div>
114
115     <!-- Dynamic Highlight Compile Board -->
116     <div class="compile-panel" id="highlight-compile-board" style="display:
none;">
117         <div class="compile-info">
118             <h3 id="compile-selected-count">0 Rallies Selected</h3>
119             <p>Generate seamless high-quality highlight reel clips compiled
directly by FFmpeg.</p>
120         </div>
121         <div class="compile-actions">
122             <div class="form-group" style="gap: 4px; margin-right: 12px;">
123                 <label style="font-size: 0.75rem; color: var(--text-
muted);">Output File Name</label>
124                 <input type="text" id="highlight-output-name" class="text-
input" style="padding: 8px 12px; font-size: 0.85rem; width: 220px;"
value="highlights_compiled.mp4">
125             </div>
126             <button id="btn-compile-highlights" class="btn-primary">
127                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0
1-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
128                 Compile Highlights
129             </button>
130         </div>
131     </div>
132 </div>
133 </div>
134 </div>
135
136     <!-- PROCESSING OVERLAY SCREEN -->
137     <div class="processing-overlay" id="loading-overlay">
138         <div class="spinner-box">
139             <div class="double-spinner"></div>
140             <h2 id="loading-title">Processing Video Stream</h2>
141             <p id="loading-subtitle">Running structural diagnostics, validation metrics,
and optical rally indexers...</p>
142         </div>
143     </div>

```

```

144
145         <script src="/static/app.js"></script>
146     </body>
147 </html>
148

```

26. assistant (2026-06-21T14:37:25.584Z)

Perfect! Now let me check for any CSRF or allowed_video_root interactions in the context of a public deployment:

27. user (2026-06-21T14:37:26.862Z)

```

backend/config/models.py:    # When set, /api/process video_path must resolve under this
backend/config/models.py:    directory (hosted/tenant
backend/config/models.py:    # mode). Default None preserves the single-user local 'any local
backend/config/models.py:    file' workflow.
backend/config/models.py:    allowed_video_root: Optional[str] = None
backend/config/models.py:
backend/config/models.py:    # --- Chunked upload (B2, PR-2) ---
backend/config/models.py:    # Landing zone for browser uploads (assembled from parts). Per-user
backend/config/models.py:    subdirectories
backend/config/models.py:    # arrive with PR-3 identity scoping. ? In hosted mode, set
backend/config/models.py:    allowed_video_root to
backend/config/models.py:    # contain upload_dir or /api/process will reject the uploaded
backend/config/models.py:    paths.
backend/config/models.py:    upload_dir: str = "output/uploads"
--
backend/config/models.py:    Google Drive (`gdrive.mount_root`). Per-backend settings are
backend/config/models.py:    loose dicts passed to the constructor ?
backend/config/models.py:    **secrets are never stored here**, only endpoints / regions / file
backend/config/models.py:    paths / source-selectors
backend/config/models.py:    (boto3/google auth chains supply credentials). See
backend/config/models.py:    ``backend/storage/``.
backend/config/models.py:    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
backend/config/models.py:    # Output root for produced artifacts. "" => fall back to
backend/config/models.py:    output.output_dir (non-breaking).
--
backend/main.py:
backend/main.py:
backend/main.py:# CORS for the web UI. allow_origins='' with allow_credentials=True is rejected
backend/main.py:by browsers per the
backend/main.py:# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies,
backend/main.py:so credentials stay
backend/main.py:# off. The origin list is configurable via the env var above (default [""]) so
backend/main.py:a hosted deploy can
backend/main.py:# lock it to its origin; the middleware is fixed at startup.
--
backend/main.py:    CORSMiddleware,
backend/main.py:    allow_origins=cors_origins_from_env(),
backend/main.py:    allow_credentials=False,
backend/main.py:    allow_methods=["*"],
backend/main.py:    allow_headers=["*"],
--
backend/main.py:        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
backend/main.py:
backend/main.py:    allowed_root = getattr(getattr(global_config, "security", None),
backend/main.py:    "allowed_video_root", None)
backend/main.py:    try:
backend/main.py:        validate_input_path(req.video_path, allowed_root=allowed_root)
--
backend/main.py:    # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is
backend/main.py:    verified when the worker
backend/main.py:    # fetches it (a fetch miss fails the job loudly). validate_input_path above
backend/main.py:    still runs for every

```



```
backend/main.py:    # input: its null-byte guard always applies, and a configured
allowed_video_root (hosted mode)
backend/main.py-    # rejects a non-local URI as out-of-root.
backend/main.py-    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

28. assistant (2026-06-21T14:37:28.951Z)

Now let me check the `validate_input_path` function to understand the security boundary:

29. user (2026-06-21T14:37:34.435Z)

```
backend/utils/paths.py: def validate_input_path(path: str, allowed_root: Optional[str] = None) ->
None:
backend/utils/paths.py-    """Reject null bytes and (when an allowed_root is configured) out-of-
root inputs.
backend/utils/paths.py-
backend/utils/paths.py-    With no allowed_root (default, single-user local mode) only the null-
byte check applies,
backend/utils/paths.py-    so the local 'point it at any of my videos' workflow is preserved.
Setting an
backend/utils/paths.py-    allowed_root (a future hosted/tenant deployment) enforces
containment.
backend/utils/paths.py-    """
backend/utils/paths.py-    if "\\x00" in path:
backend/utils/paths.py-        raise ValueError("video_path must not contain a null byte")
backend/utils/paths.py-    if allowed_root:
backend/utils/paths.py-        resolve_under_root(path, allowed_root)
```

30. assistant (2026-06-21T14:37:39.947Z)

Let me check `resolve_under_root` to understand the path traversal checks:

31. user (2026-06-21T14:37:40.294Z)

```
backend/utils/paths.py: def resolve_under_root(path: str, root: str) -> str:
backend/utils/paths.py-    """Return the real path, asserting it is contained under `root`
(raises ValueError if not)."""
backend/utils/paths.py-    rp = os.path.realpath(path)
backend/utils/paths.py-    rr = os.path.realpath(root)
backend/utils/paths.py-    try:
backend/utils/paths.py-        contained = os.path.commonpath([rp, rr]) == rr
backend/utils/paths.py-    except ValueError: # different drives on Windows
backend/utils/paths.py-        contained = False
backend/utils/paths.py-    if not contained:
backend/utils/paths.py-        raise ValueError(f"path {path!r} escapes the allowed root
{root!r}")
backend/utils/paths.py-    return rp
backend/utils/paths.py-
backend/utils/paths.py- def validate_input_path(path: str, allowed_root: Optional[str] = None) ->
None:
backend/utils/paths.py-    """Reject null bytes and (when an allowed_root is configured) out-of-
root inputs.
backend/utils/paths.py-
backend/utils/paths.py-    With no allowed_root (default, single-user local mode) only the null-
byte check applies,
backend/utils/paths.py-    so the local 'point it at any of my videos' workflow is preserved.
Setting an
backend/utils/paths.py-    allowed_root (a future hosted/tenant deployment) enforces
containment.
backend/utils/paths.py-    """
backend/utils/paths.py-    if "\\x00" in path:
```

32. assistant (2026-06-21T14:37:42.138Z)

Now let me verify the git log for M9 and #290 to understand what was merged:

33. user (2026-06-21T14:37:43.446Z)

```
2afbadf fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks (#292)
a1bec6f fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
24b5dee docs(serving): mark $7 tracker M9 ? + changelog (PR #290) + the real M7 L4 run (22
rallies)
78d5f6a feat(serving): M9-auth ? address review (CORS via env not cwd config; alg-confusion
tests)
03077c3 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
49c082a docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness
(M11) (#277)
21c93d1 docs(serving): lock the 5 M0 gating decisions (D7-D12) + add the data-flywheel harness
(M11)
6e5ac9e feat(worker): python -m backend.worker dispatcher ? bucket-driven train/infer (#251)
6878d45 feat(storage): pluggable StorageBackend ? local + S3/GCS first-class buckets (#249)
```

34. assistant (2026-06-21T14:37:45.294Z)

Perfect! Now let me check the config.json structure to understand the default setup:

35. user (2026-06-21T14:37:46.408Z)

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/config.json
./.claude/worktrees/bhi-execpolicy/config.json
./.claude/worktrees/boxhill17/config.json
./.claude/worktrees/gap-doc/config.json
./.claude/worktrees/hardchop/config.json
./.claude/worktrees/pr-stitch/config.json
./.claude/worktrees/recdoc/config.json
./.claude/worktrees/resweep/config.json
./.claude/worktrees/tier1-windowing/config.json
./.claude/worktrees/w2-safeguard/config.json
```

36. user (2026-06-21T14:37:48.510Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
```

```

26         "default_segmenter": "gemini",
27         "use_gpu": true,
28         "motion_threshold": 0.08,
29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62             "sport": "badminton",
63             "wsl_stage_dir": "~/clips",
64             "timeout_sec": 1800,
65             "velocity_threshold": 0.02
66         }
67     },
68     "output": {
69         "default_highlights_name": "highlights.mp4",
70         "db_name": "sports_indexer.db"
71     },
72     "stitching": {
73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

I notice there's no "security" section in the default config. Let me verify by checking how the config loads when security is not present:

38. user (2026-06-21T14:37:52.339Z)

```
backend/config/loader.py:def load_config(path: str | Path | None = None) -> AppConfig:
backend/config/loader.py:    """Load configuration with the following precedence:
backend/config/loader.py:
backend/config/loader.py:    1. Built-in defaults defined in the Pydantic models.
backend/config/loader.py:    2. Values from the JSON file (if present and readable).
backend/config/loader.py:    3. (Future) environment / CLI overrides.
backend/config/loader.py:
backend/config/loader.py:    Missing file or unreadable file ? returns a fully defaulted
AppConfig.
backend/config/loader.py:    Unknown keys in the file are tolerated (extra="allow" on the
model).
backend/config/loader.py:    """
backend/config/loader.py:    cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
```

39. assistant (2026-06-21T14:37:54.287Z)

Now let me search for the full config model definition:

40. user (2026-06-21T14:37:55.038Z)

```
backend/config/models.py:class AppConfig(BaseModel):
backend/config/models.py:    """Root configuration object.
backend/config/models.py:
backend/config/models.py:    All runtime configuration should be accessed via an instance of
this class
backend/config/models.py:    (or the loader) rather than raw dict lookups.
backend/config/models.py:    """
backend/config/models.py:
backend/config/models.py:    sport: Sport = "badminton"
backend/config/models.py:    ai_provider: str = "gemini"
backend/config/models.py:    # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is
the stable flagship;
backend/config/models.py:    # the alias never deprecates underneath us. Eval reports record the
run date alongside.
backend/config/models.py:    ai_model: str = "gemini-flash-latest"
backend/config/models.py:
backend/config/models.py:    validation: ValidationConfig =
Field(default_factory=ValidationConfig)
backend/config/models.py:    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
backend/config/models.py:    output: OutputConfig = Field(default_factory=OutputConfig)
backend/config/models.py:    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
backend/config/models.py:    security: SecurityConfig = Field(default_factory=SecurityConfig)
backend/config/models.py:    logging: LoggingConfig = Field(default_factory=LoggingConfig)
backend/config/models.py:    storage: StorageConfig = Field(default_factory=StorageConfig)
backend/config/models.py:    deployment: DeploymentConfig =
Field(default_factory=DeploymentConfig)
backend/config/models.py:    billing: BillingConfig = Field(default_factory=BillingConfig)
backend/config/models.py:
backend/config/models.py:    # Allow extra keys during transition so existing config.json files
do not break.
backend/config/models.py:    model_config = {
backend/config/models.py:        "extra": "allow",
backend/config/models.py:        "validate_assignment": True,
backend/config/models.py:    }
backend/config/models.py:
backend/config/models.py:    # Temporary migration shim for legacy dict-style access
(config.get("section", {}).get(...)).
backend/config/models.py:    # Will be removed once all call sites (validators, segmenters,
etc.) move to typed
```

41. assistant (2026-06-21T14:37:57.248Z)

Excellent. Now let me check if there are environment variable loading examples and document how CORS allow_origins flow:

42. user (2026-06-21T14:37:58.091Z)

```
from backend.results import Failure # standardized error/result contract
from backend.reporting import emit_indexer_report
from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
from backend import billing # M8: per-job cost ledger (USD internal / INR display)
from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)

app = FastAPI(title="Sports Highlight Indexer API")

def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
    ``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
```

43. assistant (2026-06-21T14:38:00.968Z)

Perfect! Now let me verify what #290 PR actually changed by looking at the commit:

44. user (2026-06-21T14:38:01.801Z)

```
commit aac763365b56cab8ab46be5063d67e3ea56c0b5e
Author: Avi <avi.dullu@gmail.com>
Date: Sun Jun 21 19:29:24 2026 +0530
```

```
feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)

* feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)

COMPUTE_DECOUPLED_SERVING M9 (D9). Verifies the Cf-Access JWT at the app so a direct hit to
the
Cloud Run URL can't spoof the identity header, and tags the job with the verified tenant.
DEFAULT-OFF.

- backend/api/auth.py (new): verify_cf_access_token (RS256: sig + exp + iss + aud, against
the team
  JWKS) + resolve_tenant (DEFAULT-OFF ? None = local single-user; enabled ? require+verify
the
  Cf-Access-Jwt-Assertion header ? user_id [email|sub]). `jwt` (PyJWT[crypto]) is lazy-
imported so
  the base runtime works without it; the JWKS fetch is injectable so the verifier is unit-
tested
  fully OFFLINE (in-test RSA keypair + fake JWKS). Team JWKS cached with a TTL.
- SecurityConfig: edge_auth_enabled=False (default), cf_team_domain, cf_access_aud,
cors_allow_origins.
- main.py: /api/process resolves the tenant (401 on a missing/spoofed token when enabled) ?
tags
  create_job(owner_id=...); CORS origins now config-driven (default ["*"]).
- database.py: create_job gains owner_id (NULL = local single-user; uses the v6
jobs.owner_id column).
- pyproject: PyJWT[crypto] in [dev] (tests/CI) + a production [auth] extra.

tests/test_edge_auth.py: accept + every reject path (expired / wrong aud / wrong iss / bad
```



```

v2 job rows survive; the
backend/infrastructure/database.py-      # ADD COLUMNs are applied idempotently so an
interrupted upgrade can re-run them.
backend/infrastructure/database.py-      self._add_column_idempotent(cursor, "ALTER TABLE jobs
ADD COLUMN policy TEXT")
backend/infrastructure/database.py-      self._add_column_idempotent(cursor, "ALTER TABLE jobs
ADD COLUMN next_poll_at TIMESTAMP")
backend/infrastructure/database.py-      cursor.execute("CREATE INDEX IF NOT EXISTS
idx_jobs_due ON jobs(status, next_poll_at)")
backend/infrastructure/database.py-      cursor.execute("PRAGMA user_version = 3")
backend/infrastructure/database.py-

```

47. user (2026-06-21T14:38:08.655Z)

```

def _migrate_to_v6(self, cursor):
    ("owner_id", "TEXT"),          # who pays (the non-owner user / tenant)
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
        "WHERE owner_id IS NOT NULL")
    owner_id: Optional[str] = None) -> bool:
    (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
    ``owner_id``
    "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
    max_frames, json.dumps(policy) if policy is not None else None, owner_id),
    owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
    "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
    (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
    "SELECT id, video_id, owner_id, compute_backend, gpu_type, gpu_active_seconds, "

```

48. assistant (2026-06-21T14:38:16.240Z)

Perfect! Now let me create a comprehensive report. Let me also verify the SPA's fetch calls don't send any custom auth headers:

49. user (2026-06-21T14:38:17.357Z)

(Bash completed with no output)

50. assistant (2026-06-21T14:38:20.878Z)

Good - the SPA sends only Content-Type. Let me now verify the HOSTING_PLAN or any deployment docs exist:

51. user (2026-06-21T14:38:22.391Z)

```

./docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md
./docs/COMPUTE_DECOUPLED_SERVING
./docs/HOSTING_PLAN.md

```

52. assistant (2026-06-21T14:38:24.438Z)

Let me check the HOSTING_PLAN to understand the deployment scenario:

53. user (2026-06-21T14:38:25.070Z)

```

1      # Hosting Plan ? khelsutra.guru
2
3      **Status:** PROPOSAL ? awaiting owner ratification.
4      **Date:** 2026-06-10 . **Constraints:** budget $10?25/mo . GCP credits available .
DigitalOcean
5      acceptable . alpha backend = owner's GPU box behind a tunnel . 10?25 alpha testers .
registrar = GoDaddy.
6
7      > **? Update (2026-06-20, ADR-013): DigitalOcean is DROPPED for COMPUTE** ? GCP is the

```

[illegible]


```

56                                     ? signed
resumable URLs
57                                     ?
58                                     GCS buckets
(uploads / reels, lifecycle ? cold)
59                                     ? pull
jobs, outbound-only
60                                     GPU box = first
`byo_worker` (PLATFORM P3)
61     ``
62
63     - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
64     bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
65     handles Gemini-path jobs entirely in-cloud.
66     - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
67     later); Access allowlist retires.
68     - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-
to-zero) .
69     GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
70     ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2
71     (zero egress) while keeping everything else GCP.
72
73     ## 4. Alternatives considered
74
75     | Option | Monthly (alpha ? beta) | Pros | Cons |
76     |---|---|---|---|
77     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
78     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
79     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP
standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
80     | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
81
82     ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
83
84     1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
85     imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).
86     2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
87     "I'll use my own nameservers"** ? paste the two Cloudflare nameservers ? Save.
Propagation:
88     minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
89     transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
90     3. Records (I manage after takeover): `app` ? Pages project; `api` ? `**Cloudflare API token** (you create: dash ? My Profile ? API Tokens) | Zone:DNS>Edit,

```

Account:Pages:Edit, Account:Access:Apps:Edit, Account:Tunnel:Edit ? scoped to this zone | DNS records, Pages deploys (wrangler), Access policies, tunnel config |

98 | ****GitHub repo**** `khelsutra` (private) | Pages connects once (dashboard, you; build root = `web/`) ? every push auto-deploys | I write the SPA in your Projects folder; commits pushed = deployed |

99 | ****On the box**** (one-time, I hand you exact commands) | `cloudflared` installed as a Windows service + `config.yml` | The tunnel itself |

100 | ****GCP (beta only)**** | A `khelsutra` project + service account (Cloud Run admin, Storage admin, Cloud SQL admin) | Control-plane deploys |

54. assistant (2026-06-21T14:38:27.135Z)

Perfect! Now I have all the information I need. Let me compile my findings into a structured report.

55. user (2026-06-21T14:39:06.228Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 7735 bytes): {"summary": "The FastAPI backend (badminton-highlight-indexer) has a ****production-ready CORS setup but default-OFF edge-auth****. CORS origins are env-driven and configurable; the Cf-Access edge-auth mo
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

56. assistant (2026-06-21T14:39:07.853Z)

Let me fix the JSON escaping:

57. user (2026-06-21T14:39:28.656Z)

Structured output provided successfully

58. assistant (2026-06-21T14:40:05.506Z)

Perfect! Now let me provide a comprehensive narrative summary with the key file citations:

Investigation Complete

I've systematically examined the badminton-highlight-indexer and khelsutra repos for the alpha-shareable cross-origin deployment scenario. Here are my findings:

****CORS Setup: Production-Ready, Env-Driven****

****File:**** `backend/main.py:48-68`

The FastAPI app configures CORS with:

```
```python
app.add_middleware(
 CORSMiddleware,
 allow_origins=_cors_origins_from_env(), # env var RALLY_CORS_ALLOW_ORIGINS (comma-
separated)
 allow_credentials=False, # required: wildcard + credentials rejected by
browsers
 allow_methods=["*"],
 allow_headers=["*"],
)
```
```

****Gap:**** The `RALLY\_CORS\_ALLOW\_ORIGINS` env var is not auto-populated in local dev?it defaults to `["\*"]`. For a production hosted deployment pointing a SPA at `https://app.khelsutra.guru`,

```
the operator must set:
...
RALLY_CORS_ALLOW_ORIGINS=https://app.khelsutra.guru
...

---
```

****Edge-Auth (M9, #290): Fully Merged, Default-OFF****

****Files:****

- `backend/api/auth.py` (128 lines): RS256 JWT verification against Cloudflare team JWKS
- `backend/config/models.py`: `SecurityConfig` class with 3 knobs
- `backend/main.py:294`: Tenant resolution at `/api/process` entry point
- `backend/infrastructure/database.py`: `owner\_id` column (v6 migration)

****The Config Knobs** (all default-OFF):**

| Config | Type | Example | Purpose |
|------------------------------|------|---------------------------------------|---|
| ----- ----- ----- ----- | | | |
| `security.edge_auth_enabled` | bool | `False` | Master switch (locally: off; cloud: on) |
| `security.cf_team_domain` | str | `https://myteam.cloudflareaccess.com` | Team's issuer + JWKS endpoint |
| `security.cf_access_aud` | str | `<app-uuid>.cloudflareaccess.com` | App's audience tag (from CF dashboard) |

****Operational Requirements to Flip it ON:****

1. ****CF Team Domain**** (public, cloudflareaccess.com URL): Available from Cloudflare Access dashboard once a team is created?no secrets.
2. ****Access App AUD**** (public audience string): Generated when you create a CF Access application?no secrets.
3. ****Ingress Lock**** (deployment-time): Cloud Run must reject direct hits not proxied through CF Tunnel. This requires:
 - Deploying the FastAPI on Cloud Run
 - Creating a Cloudflare Tunnel pointing to the Cloud Run URL
 - Locking Cloud Run's ingress to only accept from the Tunnel (GCP policy)
 - Configuring CF Access to gate the Tunnel

****What Happens ON vs OFF:****

- ****OFF (default):**** `resolve\_tenant(headers, config)` returns `None`; job tagged with `owner\_id=NULL` (local single-user).
- ****ON:**** Requires `Cf-Access-Jwt-Assertion` header; verifies RS256 signature + exp + iss + aud against team JWKS; returns `user\_id` (email or sub claim); job tagged with `owner\_id=<email>`.

****Verification Code**** (`backend/api/auth.py:67-99`):

- RS256 signature check
- Token expiration validation
- Issuer must match `team\_domain`
- Audience must match `cf\_access\_aud`
- Kid in header must exist in JWKS

****SPA-to-Engine Authentication: No Custom Headers****

****File:**** `backend/api/static/app.js`

```
All fetch calls (lines 88, 329, 373) send:
```javascript
headers: { "Content-Type": "application/json" }
// No Authorization, Cf-Access, or credentials fields
```
```

****How Auth Works in Production:****

1. User hits `https://app.khelsutra.guru` (Cloudflare Pages + Access)
2. CF Access shows email OTP or SSO gate ? redirects to SPA if authenticated
3. CF Access injects a secure cookie (or the JWT in a header intercepted server-side)
4. SPA makes fetch to `https://api.khelsutra.guru/api/process`
5. Cloudflare Tunnel (running on the GPU box) forwards request to FastAPI `:8000`
6. FastAPI reads `Cf-Access-Jwt-Assertion` header (injected by CF proxy, unreplicable from the browser)
7. If edge-auth enabled, verifies JWT; sets `owner\_id` on the job

Bottom Line: The SPA is **auth-agnostic**. It relies on CF Access being the auth layer, not the API itself. The API just parses the header CF injects.

****CSRF & Hosted-Mode Interactions****

File: `backend/main.py:298-311` + `backend/utils/paths.py`

CSRF: Not needed here? `allow\_credentials=False` + cross-origin means the browser won't auto-send cookies. The SPA must explicitly POST; an attacker's `` will be blocked by CORS.

Allowed Video Root (Hosted Mode):

```
python
allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root", None)
validate_input_path(req.video_path, allowed_root=allowed_root)
```

- **Unset (default):** No containment check? local mode can access any file the process user can read.

- **Set to `/uploads`:** FastAPI rejects `/etc/passwd` or `/home/user/private/video.mp4`; only allows paths under `/uploads`.

The check uses real-path normalization (`os.path.realpath`) + `os.path.commonpath` to detect `..` traversal.

Gap in Hosted Mode: If the SPA is public and `allowed\_video\_root` is not set, users can ask the engine to process any file on the box (info disclosure risk). This is intentional for single-user local mode; the owner must lock it down if going public.

****Classification: claude-doable vs Owner-Gated****

| Piece | doable | Owner-Gated | Work |
|--|--------|-------------|---|
| CORS env config | ? | ? | Set `RALLY_CORS_ALLOW_ORIGINS=https://app.khelsutra.guru` at deploy-time |
| SPA API-base URL | ? | ? | Already uses relative paths (`/api/process`); works cross-origin if CORS is set |
| allowed_video_root config | ? | ? | Set in config.json or env to lock down file access in hosted mode |
| edge_auth_enabled flag | ? | ? | Flip to `True` in config.json |
| cf_team_domain + cf_access_aud | ? | ? | Owner retrieves from Cloudflare dashboard |
| CF Tunnel + Access ingress lock | ? | ? | Owner deploys Tunnel on box + locks Cloud Run policy |
| PyJWT[crypto] extra | ? | ? | `pip install -e .[auth]` if edge-auth enabled |

****Key File Line Citations****

| What | File | Lines |
|---------------------------|-------------------|-------|
| CORS middleware + env var | `backend/main.py` | 48-68 |

| CORS allow_origins helper | `backend/main.py` | 48-55 |
| edge_auth import | `backend/main.py` | 43 |
| Tenant resolution call | `backend/main.py` | 294 |
| Edge-auth module | `backend/api/auth.py` | 1-128 |
| JWT verification logic | `backend/api/auth.py` | 67-99 |
| Default-OFF check | `backend/api/auth.py` | 111-118 |
| SecurityConfig model | `backend/config/models.py` | (grep shows class + 3 knobs) |
| Database owner_id column | `backend/infrastructure/database.py` | (v6 migration) |
| Path validation | `backend/utils/paths.py` | (resolve_under_root + validate_input_path) |
| SPA fetch calls | `backend/api/static/app.js` | 88, 329, 373 (headers only Content-Type) |
| M9 PR merge commit | git | `aac7633` |